libkrun vs firecracker

1:09 am

and libkrun are both lightweight virtual machine monitors (microVMMs) designed to run isolated workloads with fast boot times, but they target different environments, design philosophies, and integration models. GitHub +2

Key Differences

- **Host & OS Support:**
  - **Firecracker:** Runs strictly on **Linux** hosts with KVM hardware acceleration. It does not support native macOS or Windows hosts.
  - **libkrun:** Is hypervisor-agnostic. On Linux it uses KVM; on macOS Apple Silicon it uses Apple's `Hypervisor.framework` directly, making it portable across developer laptops and cloud servers. X +3
- **Filesystem & Storage:**

Ask anything

- **Architecture & Embeddability:**
  - **Firecracker:** Built by AWS as a standalone service controlled via an external REST API and a separate monitoring/jailer process, geared toward massive multi-tenant cloud orchestration (like AWS Lambda or Vercel).
  - **libkrun:** Built as a single, embeddable **dynamic library** (C/Rust) designed to be packed directly inside applications or runtimes without external supporting daemons or shims. GitHub Pages docume... +4
- **Lifecycle & State Features:**
  - **Firecracker:** First-class support for **snapshots, pause, resume, and restore** workflows.
  - **libkrun:** Lifecycle operations like snapshotting, pausing, and resuming are still experimental or not yet fully supported. Celesto AI

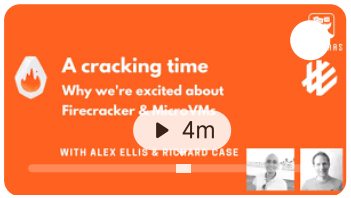
Comparison Matrix

Capability	Firecracker	libkrun
Linux Host Support	Yes (KVM)	Yes (KVM)

Filesystem Model	virtio-block (ext4 image)	virtio-fs (direct directory mount)
Snapshots & Resume	Supported	Not yet supported
Primary Use Case	Cloud-scale multi-tenant serverless & AI agent sandboxes	Local-first, cross-platform, embeddable app sandboxing

You can explore more community benchmarks and architectural breakdowns on Sandbox Benchmarks and review backend options via Celesto Backends Documentation. StarSling +1

Watch this video to learn more about how Firecracker boots microVMs directly and compares to traditional virtualization:



A cracking time: Exploring Firecracker & MicroVMs
11K views · 4 years ago
YouTube · Alex Ellis

Are you planning to run these microVMs ↪ **on Linux cloud servers** or ↪ **locally on a**

Is libkrun heavier than firecracker?

1:10 am

Slightly heavier per VM, though often lighter in overall architecture. Both tools belong

- **Device & Filesystem Overhead:** Because libkrun mounts host directories directly via `virtio-fs` and includes optional device passthrough (like GPU or sound), it carries a slightly larger device model than Firecracker's stripped-down block-device setup. iii.dev +2
- **Per-VM Density:** Firecracker is tuned specifically to pack thousands of microVMs onto a single Linux host with under **5 MiB** of VMM memory overhead per instance. As noted in Particula Tech's sizing comparison, libkrun trades a small amount of that extreme per-VM density for easier integration. Krova Cloud +1
- **Lack of CoW Snapshots:** Firecracker supports memory snapshots using `MAP_PRIVATE` copy-on-write, allowing many idle VMs to share RAM pages. Because libkrun lacks mature snapshot/restore support, scaling dozens of identical instances can consume more aggregate RAM. Celesto AI +1

Where libkrun Is Actually Lighter

- **Zero External Daemons or Shims:** Firecracker requires running a separate VMM binary, a companion `jailer` security process, and external orchestration/API plumbing per VM. As explained by the maintainers on libkrun's GitHub issue tracker, libkrun links directly into your application as a single library thread, eliminating IPC serialization and separate daemon processes. PandaStack +3
- **Cold Startup Path:** While Firecracker's core VMM boots in under 125ms, setting up its jailer, network TAP devices, and block images adds 100–300ms on a real cold path.



whereas libkrun optimizes for developer ergonomics and single-to-moderate sandbox counts.

Why Firecracker Wins at 1,000+ Scale

- **Copy-on-Write (CoW) Memory Deduplication:** lets you boot one VM, snapshot its memory to disk, and restore 1,000+ clones using `MAP_PRIVATE`. All identical instances share the same read-only host RAM pages for the guest kernel and runtime, saving tens of gigabytes of memory. libkrun does not support memory snapshots, forcing every VM to allocate its own full RAM footprint.
- **Sub-5 MiB VMM Overhead:** Firecracker's ultra-minimal device model (no `virtio-fs` daemon overhead, no GPU/display devices) keeps VMM memory overhead under **5 MiB per instance**. Across 2,000 VMs, that stays under 10 GiB of overhead.
- **Minimal File Descriptor & Thread Pressure:** Each libkrun instance relies on `virtio-fs` to map host directories, which tracks open host file descriptors and FUSE requests per VM. At 1,000+ VMs, host `ulimit` and filesystem cache contention become serious bottlenecks compared to Firecracker's read-only shared `ext4` root images.
- **Defense-in-Depth Isolation (`jailer`):** Running thousands of untrusted workloads on one host increases blast-radius risk. Firecracker's `jailer` binary automatically wraps every microVM in `seccomp` filters, cgroups, and chroot/namespace jails—features you would have to build yourself around a libkrun host process.

- **Operating System:** Bare-metal Linux (or nested-virtualization-enabled cloud metal) with KVM enabled.
- **Kernel Tuning:** Increase host limits for `vm.max_map_count`, `fs.file-max`, PID limits (`_max`), and network bridge/TAP device limits.
- **Storage:** Fast NVMe storage for snapshot memory files and base `ext4` rootfs images, paired with per-VM overlay/diff drives or ephemeral RAM disks.

If you'd like to plan the host specs, let me know:

- The **average RAM and vCPU** each microVM needs
- Whether the VMs are **short-lived bursts** or **long-running**

I can help estimate the bare-metal server sizing and kernel tuning parameters you'll need.

Then what do we do about networking and disk d for fire cracker

9:30 am

Instead of duplicating a 1–5 GB `ext4` image 1,000 times, you share a single read-only base image and give each microVM a tiny, disposable write layer:

- **Shared Read-Only Base + CoW Overlay (`dm-thin` / `overlayfs`):**
  - **Device Mapper (`devicemapper` thin provisioning):** Create one thin pool on an NVMe drive, put your base `ext4` rootfs in it, and take 1,000 instant thin snapshots. Each VM gets its own `/dev/mapper/snap-X` block device, reading shared blocks from the base and only writing changed blocks. (This is what Firecracker-containerd uses).
  - **In-Guest `overlayfs` (Fastest for ephemeral VMs):** Attach two drives to Firecracker:
    - `/dev/vda`: A single **read-only** `ext4` base image shared directly across all 1,000 VMs (`is_read_only: true`).
    - `/dev/vdb`: A tiny, sparse per-VM writable drive (e.g., a 64 MB sparse file on `tmpfs` or NVMe) or pure guest RAM (`tmpfs`), merged inside the guest via an `init` script using `overlayfs`.
- **Built-in IOPS Rate Limiting:** Configure Firecracker's `rate_limiter` token buckets on each drive so one VM running `npm install` or compiling code cannot saturate your NVMe queue for the other 999 VMs.

2. Networking Strategy (Avoiding Bridge Bottlenecks)



- Each VM gets a dedicated `tapN` device on the host with a tiny point-to-point subnet (e.g., Host `10.0.0.1/31` ↔ Guest `10.0.0.2/31`).
- Enable IP forwarding (`net.ipv4.ip_forward=1`) and `iptables`/`nftables` NAT on the host, or route subnets directly via BGP/CNI.
- **Pre-allocate TAP pools:** Creating a TAP device takes ~10–20ms. Maintain a warm pool of pre-created `tap0`..`tap1500` devices and namespaces so VM boot times stay sub-100ms.

- **Network Namespaces (`netns`) for Snapshot Clones:**
  - If you restore 1,000 VMs from the **same memory snapshot**, every restored VM wakes up thinking it has the exact same IP and MAC address (e.g., `172.16.0.2`).
  - Place each Firecracker process inside its own Linux **network namespace** (`ip netns`) with identical internal IP/MAC settings, and use `veth` + NAT (or `tc` / eBPF IP rewriting) between each `netns` and the host. This lets 1,000 snapshot clones run simultaneously with zero IP collisions.

- **eBPF (`Cilium` / `tc-bpf`):** At multi-thousand scale, replace `iptables` rules (which slow down linearly as rules grow) with eBPF packet forwarding on the TAP interfaces for O(1) packet routing and isolation.

Recommended Architecture Summary



Ephemeral Writes	Host `tmpfs` sparse files (for short-lived jobs)	Avoids NVMe wear and disk I/O contention
Network Topology	Per-VM `netns` + Point-to-Point TAP + NAT/eBPF	Eliminates Linux bridge lock & allows identical snapshot IPs
Noisy-Neighbor Control	Firecracker block & network `rate_limiter`	Caps bandwidth/IOPS per microVM in the VMM

Are you planning to build your own orchestrator around Firecracker, or would you like to know which **existing open-source frameworks** (like **Firecracker-containerd**, **Flintlock**, or **E2B**) already automate this networking and CoW storage plumbing?

Can we mount a Read only file into docker but use overlay to write to it making the write never show up on the host?

9:38 am



How to Set It Up

Because Linux `overlayfs` operates on **directories** rather than individual files, choose one of two approaches depending on whether you want Docker or the container itself to manage the overlay:

Option 1: Mount the Host Directory Read-Only & Run `overlayfs` Inside Docker (Cleanest for Ephemeral Containers)

1. Mount the host folder containing your file as read-only (`:ro`), alongside an in-memory `tmpfs` volume for the scratch space:

bash

```
docker run --rm -it \
  --cap-add SYS_ADMIN \
  --security-opt apparmor=unconfined \
  -v /path/to/host/dir:/lower:ro \
  --tmpfs /overlay:rw,exec \
  ubuntu bash
```

Use code with caution.

2. Inside the container, create the `upper` and `work` directories on the `tmpfs` and

```
  -o lowerdir=/lower,upperdir=/overlay/upper,workdir=/overlay/work \
  /merged
```

Use code with caution.

3. **Result:** When your app modifies `/merged/your-file`, Linux copies the file up into `/overlay/upper` (in RAM) and edits it there. `/lower/your-file` on the host stays strictly read-only, and when the container exits, `/overlay` vanishes.

Option 2: Create the Overlay on the Host First (No Extra Container Privileges Needed)

If you want to keep your container completely unprivileged (no `--cap-add SYS_ADMIN`):

1. Create a temporary `tmpfs` or disposable folder on the host and mount the overlay there:

bash

```
sudo mount -t overlay overlay \
  -o lowerdir=/path/to/host/dir,upperdir=/dev/shm/c1_upper,workdir=/dev
  /dev/shm/c1_merged
```

```bash
docker run --rm -v /dev/shm/c1_merged/your-file:/app/your-file ubuntu
```

Use code with caution.

3. **Result:** The container sees a normal writable file at `/app/your-file`, has zero special privileges, and all writes stay trapped in `/dev/shm/c1_upper` until you unmount and delete it.

Key Caveats to Keep in Mind

- **Copy-Up Latency on Large Files:** The first time the container writes even 1 byte to the file, `overlayfs` copies the **entire file** from `lowerdir` into `upperdir`. Keep files small (or use block-level `devicemapper` snapshots if you are exposing multi-gigabyte disk images).
- **Hard Links & Inodes:** Because copy-up creates a new inode in `upperdir`, tools that track inode numbers across writes may notice the inode change on first write.
- **Security Note (Option 1):** Granting `--cap-add SYS_ADMIN` lets a process inside the container perform mounts. For untrusted workloads, prefer **Option 2** (mounting on the host) or Firecracker microVMs.

What of wole directories

9:41am

Yes—and it's actually easier for whole directories than single files, because Linux `overlayfs` is natively designed around directories.

You can mount a host directory read-only (`lowerdir`), layer an ephemeral RAM disk (`upperdir`) on top of it, and let the container create, modify, rename, or even delete any files inside that directory without a single change touching the host.

Two Ways to Do It

1. Unprivileged Container (Recommended: Mount on Host First)

This keeps your Docker container completely locked down with zero extra Linux capabilities.

1. **Create the overlay on the host** (using `/dev/shm` for fast in-memory storage):

bash

```
mkdir -p /dev/shm/c1/{upper,work,merged}
```

2. **Bind-mount the merged directory into Docker:**

```bash
docker run --rm -it \
  -v /dev/shm/c1/merged:/app/data \
  ubuntu bash
```

Use code with caution.

3. **Cleanup after container stops:**

```bash
sudo umount /dev/shm/c1/merged && rm -rf /dev/shm/c1
```

Use code with caution.

2. Self-Contained Inside Docker (Requires `SYS_ADMIN`)

If you don't want a host wrapper script, mount the host directory read-only into the container and assemble the overlay inside the container's entrypoint:

```
--tmpfs /overlay:rw,exec \
ubuntu bash -c "
  mkdir -p /overlay/upper /overlay/work /app/data && \
  mount -t overlay overlay -o lowerdir=/lower,upperdir=/overlay/upper,w
  exec bash
"
```

Use code with caution.

How Whole-Directory Operations Behave

- **New files (`touch /app/data/new.txt`):** Created directly in `/overlay/upper` (RAM). The host never sees them.
- **Modified files (`echo hi >> /app/data/existing.txt`):** Triggers a copy-up of that specific file into `upperdir`, then edits the copy.
- **Deleted files (`rm /app/data/old.txt`):** `overlayfs` creates a special "whiteout" character device (`0/0`) in `upperdir` that hides the file from the container while leaving `/path/to/host/dir/old.txt` completely intact on the host.
- **Scaling to 1,000+ instances:** You can point **1,000 separate overlays** at the exact same `lowerdir=/path/to/host/dir` simultaneously. Every instance gets its own isolated `upperdir` and `work` folder while sharing the host's page cache for all read-

I am thinking of mounting a whole git repository from the host into docker but use worktree in docker to create disposable ...

9:42 am

Yes, this works cleanly—as long as you pair the read-only host repository with an `overlayfs` (or `GIT_COMMON_DIR` + disposable worktree) setup so `git worktree add` can write its administrative metadata.

Normally, running `git worktree add` inside a container against a strictly read-only `:ro` host `.git` directory fails because Git must create a tracking folder inside `.git/worktrees/<name>` and lock files during checkout. Combining `overlayfs` with `git worktree` solves this completely.

Why `OverlayFS + git worktree` Is Ideal at Scale

- **Zero Object Duplication:** All 1,000 containers read the multi-gigabyte `.git/objects` packfiles directly from the host's shared page cache.

How to Set It Up

Step 1: Prepare on the Host (Or Entrypoint)

Mount your host git repo as the `lowerdir` of an overlay, and point `git worktree add` to an ephemeral `tmpfs` workspace directory:

bash

```
docker run --rm -it \
  --cap-add SYS_ADMIN \
  --security-opt apparmor=unconfined \
  -v /path/to/host/repo:/repo-base:ro \
  --tmpfs /scratch:rw,exec \
  ubuntu bash -c '
    # 1. Layer a writable tmpfs over the read-only host repo
    mkdir -p /scratch/git-upper /scratch/git-work /repo-shared /workspace
    mount -t overlay overlay \
      -o lowerdir=/repo-base,upperdir=/scratch/git-upper,workdir=/scratch/
      /repo-shared

    # 2. Create a detached worktree in /workspace (also in tmpfs)
    git -C /repo-shared worktree add --detach /workspace HEAD
```

(Note: If you want unprivileged containers without `SYS_ADMIN`, run steps 1–2 in a small host script under `/dev/shm/c1/` and simply bind-mount `/dev/shm/c1/workspace` and `/dev/shm/c1/repo-shared/.git` into Docker.)

---

3 Critical Gotchas to Avoid

1. **The `.git` File Pointer Inside Worktrees:**
   In a Git worktree, `/workspace/.git` is **not** a directory—it is a plain text file containing:

   ```
   text

   gitdir: /repo-shared/.git/worktrees/workspace
   ```

   Use code with caution.

   If you create the worktree on the host and mount only `/workspace` into Docker, Git will fail inside the container unless `/repo-shared/.git` is mounted at the **exact same path** inside the container.

2. **Checkout Speed on Large Repos (`sparse-checkout`):**
   Even though `.git/objects` is shared, `git worktree add` still materializes every tracked working file into `/workspace` (consuming RAM in `tmpfs` and CPU time). If

Use code with caution.

3. **Host `git gc` Safety:**
   Never run `git gc --prune=now` on the host repository while active containers are reading from it, or you risk deleting packfiles currently memory-mapped by running containers.

Do you plan to **run untrusted user/AI-generated code** inside these worktrees, or is this for **trusted CI/test runners**? Let me know if you want to compare the security boundaries of this Docker setup vs. passing the same overlay into Firecracker.

Ok can this work with furecracker too ?

9:44 am

Yes—and it works even more cleanly with Firecracker because you avoid `virtio-fs` file-descriptor overhead and get full hardware-level VM isolation.

Instead of bind-mounting a directory, you attach **three drives** (or two drives + guest RAM) to each Firecracker microVM:

1. `/dev/vda` **(Root OS Image — Read-Only):** Shared across all 1,000 VMs (`is_read_only: true`).
2. `/dev/vdb` **(Bare Git Repo Image — Read-Only):** A single `ext4` filesystem image containing your bare repository (`repo.git`), attached simultaneously to all 1,000 VMs with `is_read_only: true`.
3. **Ephemeral Scratch Space (Guest `tmpfs` or Sparse `/dev/vdc`):** Each VM has its own isolated RAM disk (`tmpfs`) or thin-provisioned scratch block device.

Step-by-Step Setup

1. On the Host: Build the Read-Only Git Block Image Once

Create an `ext4` image containing a **bare** clone of your repository (a bare repo avoids wasting space on an unused working tree):

bash

```
# 1. Create a bare clone on the host
git clone --bare /path/to/host/repo /tmp/repo.git
git -C /tmp/repo.git repack -a -d --write-bitmap-index
```

Use code with caution.

(You can attach `/var/lib/firecracker/git-repo.ext4` with `"is_read_only": true` to 1,000+ Firecracker instances simultaneously—the host Linux kernel caches the read-only blocks in RAM once across every VM.)

2. Inside Each Firecracker MicroVM: Overlay + Worktree

Inside the microVM's `init` script (which takes ~10–15ms), mount the read-only drive, layer `overlayfs` over it in guest RAM (`tmpfs`), and check out a disposable worktree:

bash

```
#!/bin/sh
# 1. Mount the shared read-only bare repo block device
mkdir -p /mnt/repo-ro
mount -o ro,noload /dev/vdb /mnt/repo-ro

# 2. Layer a writable guest tmpfs overlay on top
mkdir -p /tmp/git-upper /tmp/git-work /mnt/repo-shared /workspace
mount -t overlay overlay \
  -o lowerdir=/mnt/repo-ro,upperdir=/tmp/git-upper,workdir=/tmp/git-work \
  /mnt/repo-shared
```

Why This Beats Docker OverlayFS at 1,000+ Scale

Feature	Docker + Host OverlayFS	Firecracker + Read-Only Block Image
Security Isolation	Shared host kernel (container escape risk)	Hardware KVM boundary (dedicated guest kernel)
Snapshot Compatibility	Containers cannot be paused/cloned mid-state	You can snapshot the VM **after** `git worktree add` and restore 1,000 clones in ~5ms
Updating the Repo	Modifying host repo under live overlays can cause stale file handles	Build `git-repo-v2.ext4` atomically; new VMs mount `v2` while running VMs stay safely on `v1`
Host Kernel Pressure	1,000 overlay mounts + thousands of open dentry/inode caches on host	Only **1** open file descriptor on the host (`git-repo.ext4`) across all 1,000 VMs

Pro Tip: Snapshot After Checkout

If all 1,000 microVMs start from the same commit, boot **one** Firecracker VM, run the `mount` and `git worktree add` commands above, and take a **Firecracker memory**

Do you plan to run untrusted user/AI-generated code inside these worktrees, or is this for trusted CI/test runners? Let m...

9:51 am

Noted—since you are running both untrusted AI/user-generated code and trusted CI runners in worktrees that push back to a central remote, **is the safer choice**, and **you do not need to rebuild the `ext4` image on every merge.**

Instead, use the read-only `ext4` image as a **base object store** (updated once a day or weekly) and let each microVM run a fast `git fetch` for the tiny delta of recent commits.

---

1. Security Boundaries: Docker + OverlayFS vs. Firecracker

Because untrusted AI-generated code can attempt arbitrary syscalls, kernel exploits, or credential theft, the isolation gap between Docker and Firecracker is critical:

Kernel Boundary	**Shared host kernel.** A single kernel zero-day or `io_uring`/syscall bug compromises the host and all 1,000 containers.	**Dedicated guest kernel per VM.** Hardware-enforced KVM virtualization isolates every workload.
Mount / `SYS_ADMIN` Risk	Running `mount -t overlay` *inside* the container grants `SYS_ADMIN` (high container-escape risk). You must assemble overlays on the host first.	`mount -t overlay` runs strictly inside the guest kernel; a malicious script cannot touch host mounts.
Credential Isolation (Git Push)	Host secrets or SSH agents mounted into containers risk leakage if namespaces are misconfigured.	Pass short-lived, branch-scoped Git tokens over `vsock` or metadata API; revoke immediately after push.
Blast Radius for Trusted CI	If untrusted AI code and trusted CI share the same Docker host, a host compromise leaks CI secrets/signing keys.	Even if an untrusted AI VM is fully pwned inside its guest OS, your host and trusted CI VMs remain isolated.

2. Solving Frequent Merges Without Rebuilding `ext4`

1. **Nightly / Hourly Cron on Host:** Rebuild `git-base.ext4` once every few hours (or daily) containing 99.9% of the repository history.
2. **At VM Boot (50–150ms):**
   - The microVM mounts `git-base.ext4` read-only and layers its `tmpfs` overlay on top.
   - Before creating the worktree, it runs a narrow fetch against your local Git mirror/cache on the host:

```bash
git --git-dir=/mnt/repo-shared fetch --depth=1 origin main
git --git-dir=/mnt/repo-shared worktree add --detach /workspace FETC
```

Use code with caution.

   - **Why it's fast:** Because 99.9% of Git objects already exist locally on `/dev/vdb`, Git negotiates only the handful of commits merged since the last nightly `ext4` build—transferring kilobytes in milliseconds.

Pattern B: `GIT_ALTERNATE_OBJECT_DIRECTORIES` (Zero Overlay Required on `.git`)

Instead of overlaying the whole bare repository, keep `/dev/vdb` mounted strictly read-

```
# 1. Mount read-only base repo
mount -o ro,noload /dev/vdb /mnt/base-repo

# 2. Initialize a tiny writable repo in guest tmpfs
git init --bare /tmp/vm-repo
echo "/mnt/base-repo/objects" > /tmp/vm-repo/objects/info/alternates

# 3. Fetch only new commits from local host mirror & checkout worktree
git --git-dir=/tmp/vm-repo remote add origin http://169.254.169
git --git-dir=/tmp/vm-repo fetch --depth=1 origin main
git --git-dir=/tmp/vm-repo worktree add --detach /workspace FETCH_HEAD
```

Use code with caution.

- **Why this shines:** You avoid `overlayfs` whiteout/copy-up overhead on Git packfiles entirely, every VM gets a clean writable `.git` folder in RAM, and pushes (`git push origin HEAD:refs/heads/ai-task-123`) send only the new commit objects back to your remote.

Pattern C: Local Host Git Proxy (`gitea` / `git-daemon` / `nginx` cache)

Point `origin` inside the microVM to an internal host endpoint (over the VM's point-to-point TAP interface) rather than GitHub/GitLab directly:

Would you like a sample ↪ **host-side Git proxy / ACL setup** (to restrict untrusted AI VMs from pushing to protected branches) or a ↪ **Firecracker snapshot script** that pre-warms the `alternates` repo before taking the snapshot?

I like pattern B, any downsides ?

9:55 am

Pattern B (`objects/info/alternates`) is very fast and avoids `overlayfs` overhead entirely, though it has four specific gotchas you need to handle in your VM setup.

1. Missing Refs & Shallow History (`fetch` Negotiation Overhead)

- **The Downside:** A brand-new `git init --bare /tmp/vm-repo` has **no** `refs/` (`refs/heads/*` or `refs/tags/*`), even though `objects/info/alternates` gives it access to all the underlying blobs and trees on `/dev/vdb`. When the VM runs `git fetch`, Git cannot tell the remote server *"I already have commit* `abc1234` *"* unless `/tmp/vm-repo` knows what commit hashes it has. Without `refs/`, Git may re-

```
cp /mnt/base-repo/packed-refs /tmp/vm-repo/packed-refs 2>/dev/null || t
```

Use code with caution.

Now `git fetch` immediately knows the exact commit tips baked into the `ext4` image and negotiates only the true delta.

2. Push Packfile Bloat (`--thin` Pack Issues)

- **The Downside:** When the microVM runs `git push`, Git calculates which objects the remote server lacks. Because `/tmp/vm-repo` treats `/mnt/base-repo/objects` as an alternate object store, Git generally avoids re-sending base objects—*unless* the remote server does not advertise those base refs (for example, if you push to a fresh branch on a proxy that lacks full refs), in which case Git might pack and upload large historical objects from `/dev/vdb`.
- **The Fix:** Ensure your host Git proxy/mirror holds the full repository refs so `git-receive-pack` advertises the common ancestor commits during `git push`.

3. Guest RAM Usage for Checked-Out Files

- **The Downside:** Even though `/mnt/base-repo/objects` stays on the shared read-only block device (`/dev/vdb`), creating a worktree at `/workspace` materializes all

- Alternatively, back `/workspace` with a thin-provisioned NVMe scratch drive (`/dev/vdc` via `dm-thin`) instead of `tmpfs` if your working trees are large.

4. Never Run `git gc` or `git repack` Inside the VM

- **The Downside:** Running `git repack -a` inside `/tmp/vm-repo` will copy every object from `/mnt/base-repo/objects` into `/tmp/vm-repo/objects`, defeating the shared storage and spiking guest memory/disk usage.
- **The Fix:** Disable auto-gc in `/tmp/vm-repo/config` during initialization:

```bash
git --git-dir=/tmp/vm-repo config gc.auto 0
```

Use code with caution.

Summary of the Hardened Pattern B Init Script

```bash
#!/bin/sh
mount -o ro,noload /dev/vdb /mnt/base-repo
```

```
git --git-dir=/tmp/vm-repo fetch --depth=1 origin main
git --git-dir=/tmp/vm-repo worktree add --detach /workspace FETCH_HEAD
```

Use code with caution.

Roughly how large is your repository's **working tree** (checked-out source files excluding `.git`)? Knowing that will help determine whether `tmpfs` in guest RAM or a thin-provisioned NVMe scratch drive is better for `/workspace`.